Midterm project: Block Bootstrap to estimate the downfall of an investment portfolio

Introduction

Stock investors commonly hold portfolios containing a combination of exchange-traded funds (ETFs) and individual stocks. Understanding the risks associated with these portfolios is essential for making informed investment decisions.

Traditional financial risk analysis often relies on measures such as historical volatility (eg. beta, which measures a stock's volatility or systematic risk compared to the broader market) or standard deviation. Although useful, these measures have important limitations. Financial returns can exhibit volatility clustering, extreme losses, and dependence across assets, particularly during market downturns.

Thus, the central question motivating this project is: **Given a portfolio's composition and historical market behavior, how severe could its future losses become?**

For the midterm part, I'm going to use three most common ETF to demonstrate my method: QQQ (tracks NASDAQ), VOO (tracks SP500), SCHD (a famous ETF by Charles Schwab that includes some best companies with high dividends).

Method

Getting the real data (Historical prices)

```
library(quantmod)
```

```
Loading required package: xts
```

```
Loading required package: zoo
```

```
Attaching package: 'zoo'

The following objects are masked from 'package:base':

    as.Date, as.Date.numeric

Loading required package: TTR

Registered S3 method overwritten by 'quantmod':
  method            from
  as.zoo.data.frame zoo
```

```r
library(ggplot2)
```

```
Warning: package 'ggplot2' was built under R version 4.5.2
```

```r
tickers <- c("VOO", "QQQ", "SCHD")
```

The `quantmod` package is used to retrieve financial market data, while `ggplot2` is used to visualize simulated portfolio outcomes.

```r
prices_list <- lapply(tickers, function(ticker) {
  x <- getSymbols(
    ticker,
    src = "yahoo",
    from = "2015-01-01",
    to = "2026-01-01",
    auto.assign = FALSE
  )
  Ad(x)
})

prices <- do.call(merge, prices_list)
colnames(prices) <- tickers

prices <- na.omit(prices)
```

The `lapply()` function iterates over the three ticker symbols and applies the same downloading procedure to each asset. The data was downloaded for 01/01/2025 to 01/01/2026.

Calculating the daily return

```
# Align dates and remove missing observations
prices <- na.omit(prices)
write.csv(prices, "prices.csv", row.names = FALSE)

# Daily simple returns
returns <- na.omit(
  prices / lag(prices, 1) - 1
)
returns <- as.matrix(returns)
```

calculate the daily sample return of each of the ETF and store the value in matrix.

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

Simulation

Function 1: `riskpath_sim()`

The `riskpath_sim()` function generates simulated future portfolio values based on historical daily asset returns. Unlike conventional bootstrap methods that independently sample individual observations, this function samples consecutive blocks of historical returns. This approach preserves short-term temporal dependence within each block and the observed cross-asset relationships.

Inputs:

- `returns`: A numeric matrix containing historical daily returns, with rows representing trading days and columns representing assets.
- `weights`: A numeric vector specifying the portfolio allocation across assets. Weights must be nonnegative and sum to one.
- `initial_value`: The initial monetary value of the investment portfolio.
- `horizon`: The number of trading days to simulate.
- `n_sim`: The number of simulated portfolio trajectories.
- `block_size`: The number of consecutive historical trading days sampled in each bootstrap block.

Output:

A numeric matrix with `horizon + 1` rows and `n_sim` columns. Each column represents one simulated portfolio trajectory, beginning with the initial portfolio value.

Implementation:

The function first validates its inputs using `stopifnot()`. It then randomly selects historical blocks with replacement, concatenates them to construct simulated return sequences, and calculates daily portfolio returns using matrix multiplication (`%*%`). Finally, `cumprod()` calculates the cumulative portfolio value over the simulation horizon.

```
# Start the function
riskpath_sim <- function(returns, weights,
                         initial_value,
                         horizon,
                         n_sim,
                         block_size) {

# Input validation - making sure that there are no errors in the return data
stopifnot(
    is.numeric(returns),
    all(is.finite(returns)),
    ncol(returns) == length(weights),
    all(weights >= 0),
    abs(sum(weights) - 1) < 1e-8,
    block_size >= 1,
    block_size <= nrow(returns),
    horizon >= 1,
    n_sim >= 1)

  #if (!is.numeric(returns))
  #stop("The variable `returns` is not numeric.")
  # assertthat package

  n <- nrow(returns)
  n_blocks <- ceiling(horizon / block_size)

# Store simulated portfolio values into a matrix. I have to add an additional line because o
  paths <- matrix(
    NA_real_,
    nrow = horizon + 1,
    ncol = n_sim
  )
```

```
  paths[1, ] <- initial_value



# Begin the simulation loop
  for (s in seq_len(n_sim)) {

# Sample starting positions for blocks (you have to have block_size consectutive days, so the
    starts <- sample.int(
      n - block_size + 1,
      n_blocks,
      replace = TRUE
    )

# Construct one simulated return sequence
    # unlist: combine everything into ONE vector
    # lapply: produce a list of returns, starting from the selected starting day, lasting n_
    idx <- unlist(lapply(
      starts,
      function(i) i:(i + block_size - 1)
    ))

# Turncate into the number of horizon
    idx <- idx[seq_len(horizon)]

# Extract the simulated daily ETF return from the real data
    sim_returns <- returns[idx, , drop = FALSE]

# Calculate the portfolio's daily return by weights
    portfolio_returns <- as.vector(
      sim_returns %*% weights
    )

# Calculate portfolio value over time by the cumulative product function *cumprod*
    paths[-1, s] <- initial_value *
      cumprod(1 + portfolio_returns)
  }

  return(paths)
}
```

Test it with VOO=50%, QQQ=30%, SCHD=20%. Suppose I have a initial value of $10,000

in my portfolio, and I want to simulate a length of 150 days, the block size (days consecutive) is 10, and I want to simulate for 5000 datasets.

```
set.seed(123)

paths <- riskpath_sim(
  returns = returns,
  weights = c(0.5, 0.3, 0.2),
  initial_value = 10000,
  horizon = 150,
  n_sim = 5000,
  block_size = 10
)
dim(paths)
```

```
[1]  151 5000
```

The dimension of the output should be horizon+1 times n_sim.

Results

Calculate the estimands of interest

First calculate the loss:

$$Loss = 1 - \frac{V_{terminal}}{V_{initial}}$$

Ranges from 0-1. The bigger indicates the more loss.

Then we calculate the esitmands of interests:

1. Value at risk (VaR): [confidence] percentage of the simulated scenarios have a terminal loss no greater than the number got.
2. Expected Shortfall (ES): The average loss among the worst simulated outcomes.
3. Maximum drawdown (MD): The average of the largest decline in portfolio value for all the simulations
4. Probability of exceeding a [threshold] percentage decline.The `riskpath_metrics()` function summarizes the simulated portfolio trajectories into interpretable financial risk measures. It allows investors to evaluate both potential terminal losses and declines occurring at any point during the investment horizon.

Function 2: `riskpath_metrics()`

The `riskpath_metrics()` function summarizes the simulated portfolio trajectories into interpretable financial risk measures. It allows investors to evaluate both potential terminal losses and declines occurring at any point during the investment horizon.

Inputs:

- `paths`: A numeric matrix of simulated portfolio values produced by `riskpath_sim()`.
- `confidence`: The confidence level used to calculate Value at Risk, defaulting to 0.95.
- `threshold`: The maximum drawdown threshold of interest, defaulting to 0.20.

Output:

A named list containing four risk measures:

1. **Value at Risk (VaR):** The specified quantile of simulated terminal portfolio losses.
2. **Expected Shortfall (ES):** The average terminal loss among simulations at or beyond the VaR threshold.
3. **Mean Maximum Drawdown:** The average of the largest peak-to-trough percentage declines across simulated trajectories.
4. **Probability of Exceeding the Drawdown Threshold:** The proportion of simulated trajectories whose maximum drawdown reaches or exceeds the specified threshold.

Implementation:

The function uses `quantile()` and `mean()` to calculate terminal loss measures. It applies `apply()` across the columns of the simulation matrix to calculate maximum drawdown separately for each trajectory. The `cummax()` function identifies the running peak of each portfolio trajectory, allowing drawdowns to be calculated efficiently.

Together, these two functions provide a reusable framework for simulating portfolio performance and quantifying potential investment losses under different portfolio configurations.

```
# Calculate
riskpath_metrics <- function(paths,
                             confidence = 0.95,
                             threshold = 0.20) {

# Terminal portfolio losses
  losses <- 1 - paths[nrow(paths), ] / paths[1, ]

# Value at Risk
```

```
  VaR <- unname(quantile(losses, confidence))

# Expected Shortfall
  ES <- mean(losses[losses >= VaR])

# Maximum decline, apply for each simulation
  MD <- apply(paths, 2, function(p) {
    max(1 - p / cummax(p))
  })

  list(
    VaR = VaR,
    ES = ES,
    mean_max_decline = mean(MD),
  # The probablity that the decline exceed the [threshhold]
    probability_20pct_decline =
      mean(MD >= threshold)
  )
}

results <- riskpath_metrics(paths)
print(results)
```

```
$VaR
[1] 0.1198031

$ES
[1] 0.179385

$mean_max_decline
[1] 0.111975

$probability_20pct_decline
[1] 0.0876
```

plots

```
# Calculate percentiles of terminal portfolio values
terminal_values <- paths[nrow(paths), ]
```

```r
percentiles <- quantile(
  terminal_values,
  probs = c(0.05, 0.25, 0.50, 0.75, 0.95)
)

# Prepare all original simulated paths
plot_data <- data.frame(
  day = rep(0:(nrow(paths) - 1), times = ncol(paths)),
  value = as.vector(paths),
  simulation = factor(rep(
    seq_len(ncol(paths)),
    each = nrow(paths)
  ))
)

# Create five straight lines from initial to terminal values
line_data <- data.frame(
  percentile = factor(
    c("5%", "25%", "50%", "75%", "95%"),
    levels = c("5%", "25%", "50%", "75%", "95%")
  ),
  start_day = 0,
  end_day = nrow(paths) - 1,
  start_value = paths[1, 1],
  end_value = as.numeric(percentiles)
)

# Create terminal value labels
line_data$label <- paste0(
  line_data$percentile,
  ": $",
  format(
    round(line_data$end_value, 0),
    big.mark = ",",
    scientific = FALSE
  )
)

# Plot all simulated paths and five straight percentile lines
ggplot() +

  # All 10,000 original simulated paths
```

```
  geom_line(
    data = plot_data,
    aes(x = day, y = value, group = simulation),
    alpha = 0.01,
    color = "steelblue",
    linewidth = 0.3
  ) +

  # Five straight lines connecting initial and terminal values
  geom_segment(
    data = line_data,
    aes(
      x = start_day,
      y = start_value,
      xend = end_day,
      yend = end_value,
      color = percentile
    ),
    linewidth = 1.2
  ) +

  # Label terminal values
  geom_text(
    data = line_data,
    aes(
      x = end_day,
      y = end_value,
      label = label,
      color = percentile
    ),
    hjust = -0.1,
    size = 3.5,
    show.legend = FALSE
  ) +

  # Initial portfolio value
  geom_hline(
    yintercept = paths[1, 1],
    linetype = "dashed",
    color = "gray50"
  ) +
```

```
  scale_color_manual(
    values = c(
      "5%" = "red",
      "25%" = "orange",
      "50%" = "black",
      "75%" = "darkgreen",
      "95%" = "purple"
    )
  ) +

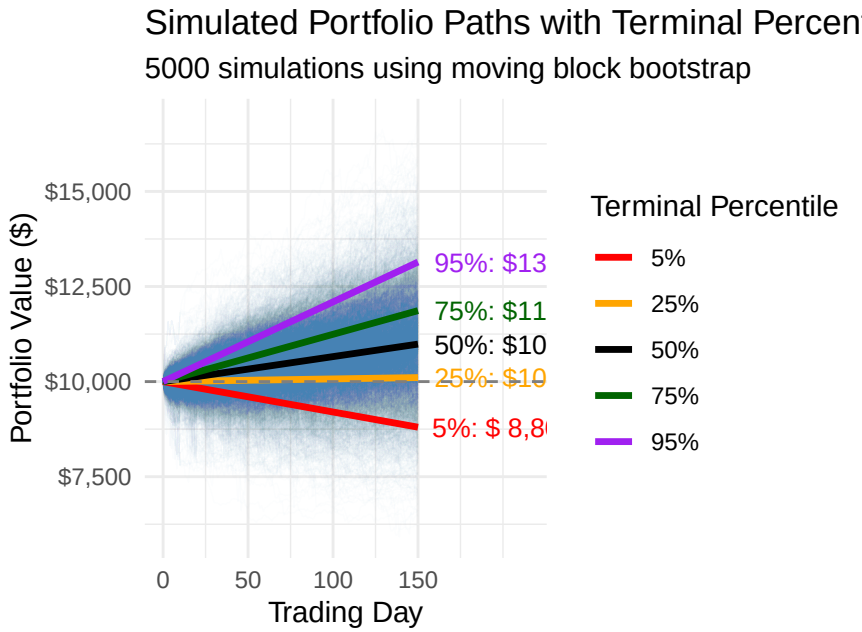
  # Extra space for terminal labels
  scale_x_continuous(
    limits = c(0, nrow(paths) - 1 + 65),
    breaks = seq(0, nrow(paths) - 1, by = 50)
  ) +

  scale_y_continuous(
    labels = scales::label_dollar()
  ) +

  labs(
    title = "Simulated Portfolio Paths with Terminal Percentiles",
    subtitle = paste(
      ncol(paths),
      "simulations using moving block bootstrap"
    ),
    x = "Trading Day",
    y = "Portfolio Value ($)",
    color = "Terminal Percentile"
  ) +

  theme_minimal() +

  theme(
    plot.margin = margin(10, 70, 10, 10)
  )
```




```
library(scales)

# Set confidence level
confidence <- 0.95

# Calculate terminal losses for all simulations
losses <- 1 - paths[nrow(paths), ] / paths[1, ]

# Calculate 95% VaR
VaR <- unname(quantile(losses, confidence))

# Count simulations with losses greater than VaR
n_exceed <- sum(losses > VaR)
pct_exceed <- mean(losses > VaR) * 100

# Create data frame
loss_data <- data.frame(
  loss = losses,
  group = ifelse(losses > VaR, "Beyond VaR", "Below VaR")
)

# Plot histogram
ggplot(loss_data, aes(x = loss)) +
```

```
  # Histogram with bars colored by VaR threshold
  geom_histogram(
    aes(fill = after_stat(x > VaR)),
    bins = 60,
    color = "white"
  ) +

  # VaR threshold line
  geom_vline(
    xintercept = VaR,
    color = "red",
    linetype = "dashed",
    linewidth = 1.2
  ) +

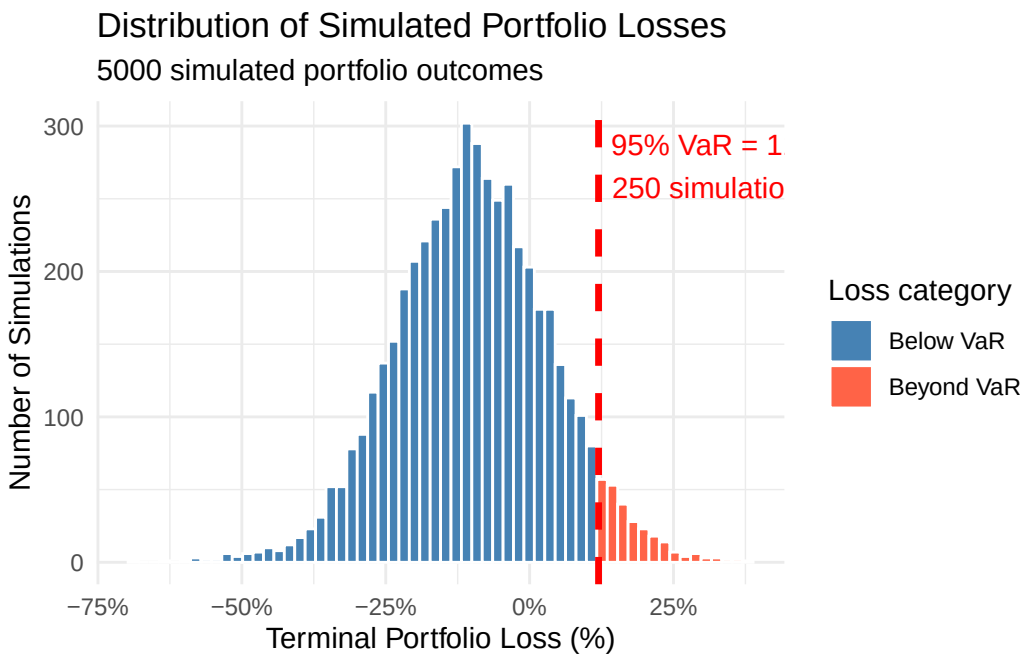
  # Label VaR threshold
  annotate(
    "text",
    x = VaR,
    y = Inf,
    label = paste0(
      "95% VaR = ", round(VaR * 100, 2), "%\n",
      n_exceed, " simulations (",
      round(pct_exceed, 2), "%)"
    ),
    hjust = -0.05,
    vjust = 1.5,
    color = "red",
    size = 4
  ) +

  scale_fill_manual(
    values = c(
      "FALSE" = "steelblue",
      "TRUE" = "tomato"
    ),
    labels = c(
      "FALSE" = "Below VaR",
      "TRUE" = "Beyond VaR"
    ),
    name = "Loss category"
  ) +
```

```
  scale_x_continuous(
    labels = percent_format(accuracy = 1)
  ) +

  labs(
    title = "Distribution of Simulated Portfolio Losses",
    subtitle = paste(
      ncol(paths),
      "simulated portfolio outcomes"
    ),
    x = "Terminal Portfolio Loss (%)",
    y = "Number of Simulations"
  ) +

  theme_minimal()
```




```
# Set the drawdown threshold
threshold <- 0.20

# Calculate maximum drawdown for each simulation
max_drawdowns <- apply(paths, 2, function(p) {
  max(1 - p / cummax(p))
```

```
})

# Calculate mean maximum drawdown
mean_mdd <- mean(max_drawdowns)

# Calculate probability of exceeding the threshold
n_exceed <- sum(max_drawdowns >= threshold)
prob_exceed <- mean(max_drawdowns >= threshold)

# Create data frame
drawdown_data <- data.frame(
  max_drawdown = max_drawdowns
)

# Plot histogram
ggplot(drawdown_data, aes(x = max_drawdown)) +

  # Histogram colored by threshold
  geom_histogram(
    aes(fill = after_stat(x >= threshold)),
    bins = 60,
    color = "white"
  ) +

  # Mean maximum drawdown
  geom_vline(
    xintercept = mean_mdd,
    color = "black",
    linetype = "dashed",
    linewidth = 1.2
  ) +

  # Threshold line
  geom_vline(
    xintercept = threshold,
    color = "red",
    linetype = "dashed",
    linewidth = 1.2
  ) +

  # Mean MDD label
  annotate(
```

```
    "text",
    x = mean_mdd,
    y = Inf,
    label = paste0(
      "Mean MDD = ",
      round(mean_mdd * 100, 2), "%"
    ),
    hjust = 1.05,
    vjust = 1.5,
    color = "black",
    size = 4
  ) +

  # Threshold probability label
  annotate(
    "text",
    x = threshold,
    y = Inf,
    label = paste0(
      "Threshold = ", threshold * 100, "%\n",
      n_exceed, " simulations (",
      round(prob_exceed * 100, 2), "%)"
    ),
    hjust = -0.05,
    vjust = 3,
    color = "red",
    size = 4
  ) +

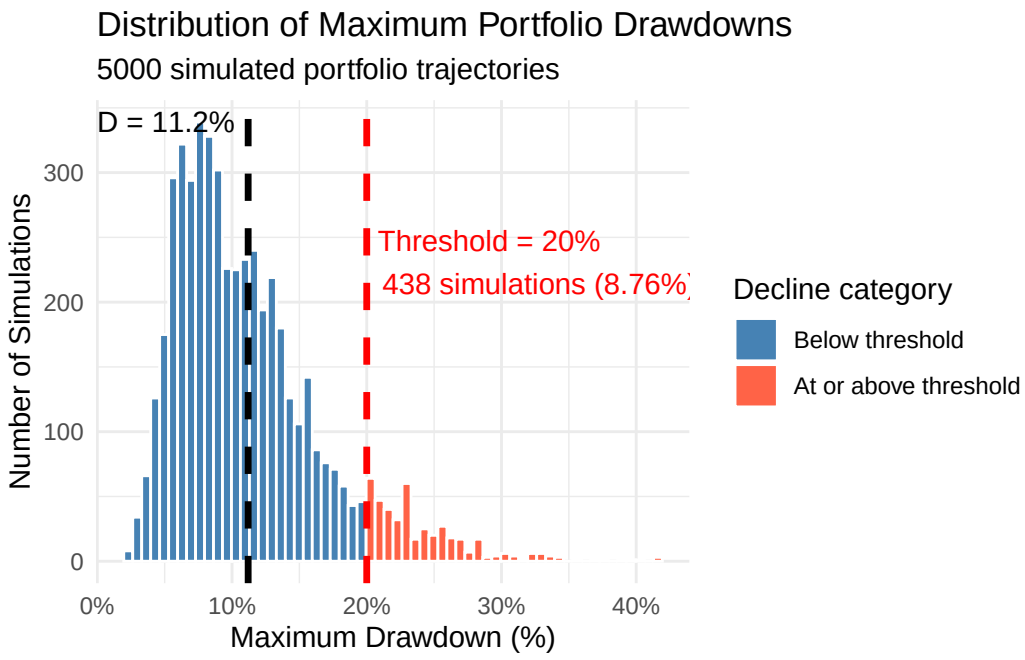
  scale_fill_manual(
    values = c(
      "FALSE" = "steelblue",
      "TRUE" = "tomato"
    ),
    labels = c(
      "FALSE" = "Below threshold",
      "TRUE" = "At or above threshold"
    ),
    name = "Decline category"
  ) +

  scale_x_continuous(
```

```
    labels = percent_format(accuracy = 1)
  ) +

  labs(
    title = "Distribution of Maximum Portfolio Drawdowns",
    subtitle = paste(
      ncol(paths),
      "simulated portfolio trajectories"
    ),
    x = "Maximum Drawdown (%)",
    y = "Number of Simulations"
  ) +

  theme_minimal()
```



AI Disclosure

1. The workflow (block bootstrap) to simulate future data was thought by me. Then, I described my process in details and used AI to generate the code.
2. Then I did a very detail breakdown and hand checked each part of the code AI generated and annotated each part in the function in details. What I did not know before is

`stopifnot`, `unlist`, and `cumprod`. I learned how these function works and made sure it follows my idea.

3. I used AI completely to generate the plots, although I already have an idea in mind the line graph and the histogram I want. AI nowadays is really good at visualization once you have the data and describe what kind of relationship you want to visualize.